The $\mu\mathcal{SZ}$ Notation
Version 1.0

Robert Büssow, Robert Geisler,
Wolfgang Grieskamp, Marcus Klar

Bericht-Nr. 97-26

The $^{-}\mathcal{SZ}$ Notation
Version 1.0

Robert Büssow, Robert Geisler, Wolfgang Grieskamp and Marcus Klar

Abstract

This is the reference manual for the $^{-}\mathcal{SZ}$ specification notation. $^{-}\mathcal{SZ}$ is a combination of statecharts and Z for the specification of safety-critical control systems. It combines statecharts' expressive power for the description of reactive behavior with Z's ability to describe data and data transformation. Additionally, Z is extended for the specification of abstract temporal properties and means to structure the specification in components are introduced. This manual covers a complete presentation of the language, including its visible syntax, a first glance at a $^{-}\mathcal{SZ}$ library, the LaTeX input syntax, and support for Statemate specifications.

Contents

1 Introduction **2**

2 Top-Level Objects **3**

3 Specifying the Data Space **5**

4 Specifying Operational Behavior **8**

5 Specifying Behavioral Constraints **14**

6 Specifying Structural Embedding **18**

A LaTeX Syntax for $\mu\mathcal{SZ}$ **20**

B LaTeX Symbols **25**

C Library **27**

D Alternative External Representations **29**

E Using Statemate **30**

1 Introduction

The ESPRESS project (Engineering of safety-critical embedded systems) aims to increase productivity in developing complex, safety-critical embedded systems and enhance the reliability of such systems by the development of a methodological tool-supported software technology. Because of the safety-criticality of the application area, formal methods are used to describe the system to be developed in order to obtain a precise mathematical description in early stages of the development. This formal specification not only serves as a basis for system design, but allows test-case generation and verification as early as the requirements phase. One crucial task is support for the separation of concern principle. The system model is divided into different views. Different aspects are described using different established specification formalisms, thus preserving the semantics of the individual formalisms.

Within the ESPRESS project, the $\mu\mathcal{SZ}$ notation has been successfully applied to industrial case studies [BGKM97, BGK97, BGK98, GHD98, CW97, Guh97]. The ESPRESS methodology provides detailed guidelines [GHD98] and tool support based on Statemate [HLN+90] for the use of $\mu\mathcal{SZ}$ in the application area.

The $\mu\mathcal{SZ}$ notation is a combination of several notations, based on Z [Spi92] and statecharts [Har87], for the formal specification of reactive systems. Within a $\mu\mathcal{SZ}$ specification a reactive system is composed in concurrent communicating processes. These processes are described by so-called process classes, which are structures with encapsulated data space and concurrent behaviour.

$\mu\mathcal{SZ}$ is designed in a modular fashion, providing four different views and related specification constructs for formulating process classes:

- Data Space – the data space of process classes is described using conventional Z.
- Operational Behavior – the reactive behavior of processes is described in an operational style using statecharts and static reactions.
- Behavioral Constraints – constraints on the reactive behavior are described in an axiomatic style using a variant of discrete temporal interval logic which extends Z.
- Structural Embedding – the embedding of a process in its environment is described by using a variant of data-flow diagrams which specify the communication topology in a network of processes. Communication is realized via shared variables.

$\mu\mathcal{SZ}$ can be used in several methodological ways and can be extended. It is possible to introduce new roles for elements of the data description—such as that a given property has to be a theorem of the specification. These roles have no effect on the basic semantics of $\mu\mathcal{SZ}$, but special methodologies and their related analysis tools can make use of them. For a detailed explanation of the role concept, see Section 3.2.

Behavioral description can be achieved in different ways in $\mu\mathcal{SZ}$, depending on the chosen methodology: by exclusively using statecharts, by exclusively using temporal logics, or by using a combination of both.

Scope This report is not intended as a tutorial introduction to $\mu\mathcal{SZ}$, but shall serve as a reference manual for the $\mu\mathcal{SZ}$ specifier. It describes the $\mu\mathcal{SZ}$ notation regarding its syntax and informal semantics. In the appendices, standard libraries for $\mu\mathcal{SZ}$ are defined, the lexical representation of $\mu\mathcal{SZ}$ under LaTeX is given, and how Statemate can be used to describe parts of the $\mu\mathcal{SZ}$ model is explained.

However, this report should also be useful for readers familiar with the constituting notations of $\mu\mathcal{SZ}$ so they can get an overview of the conceptual design.

Notation Wave-underlined symbols appearing in this report refer to Spivey's Z Syntax [Spi92, p.143ff]. Sometimes, parts of the original Z syntax are conservatively extended by adding further variants (which is denoted by a syntax rule such as symbol ::= ... | *new variants*).

2 Top-Level Objects

Specification ::= (Paragraph | Process-Class)*

Process-Class ::=
Class-Name [Gen-Formals]
Class-Paragraph*

Class-Paragraph ::= Data-Paragraph
| Operational-Paragraph
| Behavioral-Paragraph
| Structural-Paragraph

Class-Name ::= Word

A μSZ specification consists of ordinary Z Paragraphs and Process-Class definitions. Ordinary Z objects declared in the paragraphs (i.e. basic types, variables declared in axiomatic definitions, schemas and abbreviations) are visible within the paragraphs and in all process classes. Objects that are declared inside process classes are not visible in the (top-level) paragraphs. Cyclic dependencies between declarations of different paragraphs and/or process classes are not allowed.

A process class definition describes the interfaces and behavior of a set of processes.[1] It consists of a collection of four different kinds of paragraphs, the order of which is not relevant:

- Data-Paragraph: the instances of a process class, the processes, have a data space which is partitioned into private and shared variables. The data space, inclusive of operations on the data space, is described by paragraphs of this kind, which are based on conventional Z (Section 3).
- Operational-Paragraph: operational behavior of processes is described by statecharts and static reactions, which are given by paragraphs of this kind (Section 4 on page 8).
- Behavioral-Paragraph: behavioral constraints of processes can be described in an axiomatic style using an embedding of discrete temporal interval logics into Z. In fact, this embedding is a symmetric extension of the facilities for specifying the data space of a process in Z (Section 5 on page 14).
- Structural-Paragraph: the relation of a process class with other processes and the environment is defined by structural paragraphs. They describe which other classes are enriched by the given one (or from which other classes the given one inherits), and which instances of other classes are aggregated or associated by an instance of this class, inclusive of the data flow between them (Section 6 on page 18).

A process class P can be instantiated by aggregating it in another process class (see Section 6 on page 18) via a declaration of the form $p : P$. The system specified is given by an instance of a dedicated process class – the selection of this top-level instance is, however, not part of the $\mu\mathcal{SZ}$ notation.

[1] Note that the set of processes is static, e.g. creation and deletion of processes is not supported.

A process class can be generic over some types, meaning that all of its components (including Z paragraphs) are generic over these types.

To support literate specification, a class can be specified in different parts by simply partitioning it in several boxes with the same class name. In this case the list of generics, if supplied, must be the same in the different boxes.

3 Specifying the Data Space

Data-Paragraph ::= Paragraph

| | [Schema-Role [Schema-Name] ; Decl-Part / Axiom-Part]

| Schema-Role Schema-Name $\widehat{=}$ Schema-Expr

Schema-Role ::= *DATA* | *PORT* | *INIT* | *PROPERTY* | Role-Name

Role-Name ::= Word

The data space of a process class, and observations and operations over the data space, are specified by a Data-Paragraph, which may be either a conventional Z paragraph or a schema with a role.

3.1 Z Paragraphs

A conventional Z paragraph may appear in a process class definition. It serves to declare values, types and schemas that are only visible in this class or classes enriching this class. The variables introduced in axiomatic definitions are also called static variables, in contrast to dynamic variables of the class' data space. In contrast to dynamic variables, static variables do not change their value during the lifetime of a classes' instantiation. However, their values might not be completely fixed by constraints. In this case, they can have different values in different class' instantiations. We say that the class is loosely specified. Variables can be generic over some type. Generic parameters of the class itself can be interpreted as given types of the class. In contrast to given types of the class, generic parameters are actualized in instantiations of the class.

3.2 Schemas with Role

In the context of a $\mu\mathcal{SZ}$ class, a role may be attached to a schema. Such schemas cannot be generic. The role of the schema defines how it contributes to the semantics of the class. Schemas with a role do not have to be named. If they have a name, they can be referred to by their names like ordinary Z schemas.

The Data Role The variables declared in these schemas constitute the private data space of the process class. The axioms describe invariants over the values of the private variables which hold continuously for every instance of the process class. All the *DATA* schemas of a process class are collected in a schema called DATA, which is implicitly available in every process class and is defined as the conventional Z schema conjunction of the individual *DATA* schemas.

The Port Role The variables declared in these schemas constitute the public data space, i.e. the part of the data space that is visible to the environment of the class' instances. These variables can be read and/or written by other processes. The axioms describe invariants over the values of the public variables which hold continuously for all the process class' instances. Communication between processes is established by binding ports of these processes as described in Section 6. Variables with the same name of bound ports are identified (shared). Thus, a port is also a view on the public variables, giving access to these variables. From the environment of a process, public variables can be accessed only through a port. All the *PORT* schemas of a process class are collected in a schema named PORT, which is implicitly available in every process class and is defined as the conventional Z schema conjunction of the individual *PORT* schemas.

The Property Role The variables of this schema have to already be part of the data space (that is, either introduced by a *DATA* or *PORT* schema). The axioms specify invariants on their values which hold for the lifetime of a process. The difference between the *DATA*/*PORT* roles and the *PROPERTY* role is that the latter is not allowed to introduce new variables in the data space. The axiom part, however, is interpreted in a similar way. All the *PROPERTY* schemas of a process class are collected in a schema named PROPERTY, which is implicitly available in every process class. The PROPERTY schema is defined as the conventional Z schema conjunction of the individual *PROPERTY* schemas.

The Init Role The variables of this schema have to already be part of the data space (that is, either declared in a *DATA* or *PORT* schema). The axioms specify the values of the variables at the beginning of the lifetime of a process. The difference between the *PROPERTY* role and the *INIT* role is that the latter specifies a condition which has to hold exactly at the beginning of a process' lifetime (the initial state of a class' instantiation). All the *INIT* schemas of a process class are collected in a schema named INIT, which is implicitly available in every process class and is defined as the conventional Z schema conjunction of the individual *INIT* schemas.

Customized Roles To support methodological structuring of specifications, $\mu\mathcal{SZ}$ allows the introduction of new schema roles. In the basic context conditions and semantics of $\mu\mathcal{SZ}$, these roles are interpreted as aliases for one of the basic roles given above. It is also possible to define a role alias for plain schemas which do not contribute to the semantics of a process class, but will be used in a particular way in the specification (for example, to express that a schema is intended as an operation on the data space). The declaration of new roles is not part of the $\mu\mathcal{SZ}$ notation[2], but part of tools which support a certain methodology. The set of non-basic roles as currently used in the ESPRESS methodology is defined as an example in Section 3.4 on the following page.

3.3 The Data Space of a Process Class

The data space of a process is constituted by the schema

$$\text{DATASPACE} \mathrel{\widehat=} [\text{DATA};\ \text{PORT} \mid \text{PROPERTY}]$$

The schema DATASPACE is implicitly available in every process class.

Note that in general it is possible that a variable with the same name appears both in *DATA* and in *PORT* schemas, as long as its declaration is type compatible. If this is the case, public visibility dominates.

Variables belonging to the data space are not visible by default in any other schemas; schema references have to be used to incorporate them where needed. For example, below we introduce a private variable x and define its initial value. The *INIT* schema includes the *DATA* schema to gain access to x:

DATA X

$$x : \mathbb{N}$$
$$x \geq 0$$

INIT

$$X$$
$$x = 0$$

Variables of the data space are also called dynamic variables since they may change their values through the lifetime of a process. In contrast to static variables, declared in the Z paragraphs,

[2] Possibly subject to change in future versions of $\mu\mathcal{SZ}$

looseness in the description of dynamic variables is interpreted as non-determinism of the specification.

3.4 Customized Roles used in ESPRESS

For use of the $\mu\mathcal{SZ}$ notation in the ESPRESS methodology, the following customized schema roles are defined:

- *Op*: In the basic model, a plain schema. In ESPRESS, it is intended to be used as an operation on the data space. It is usually referred to in the label expression of a transition of a statechart or a static reaction (but may be referred to by a temporal predicate as well).
- *Guard*: In the basic model, a plain schema. In ESPRESS, it is intended to be used as a condition on the data space. It is usually referred to by a label expression (but may be referred to by a temporal predicate as well).
- *Assertion*: In the basic model, a *Property* schema. In ESPRESS, it is intended to represent a theorem about the specification, as for example a safety requirement. This safety requirement must be proven to be a logical consequence of the specification. Assume that all *Assertion* schemas are collected in a schema named ASSERTION. This means that if the assertions make no assumptions about the dynamic behavior, the following implication should hold:

  DATASPACE $\Rightarrow$ ASSERTION

3.5 Selective Delta

Decl-Part	::=	...
	|	Schema-Ref
Schema-Ref	::=	...
	|	Δ(Decl-Name-List)Schema-Ref

$\mu\mathcal{SZ}$ provides an extension of the Δ operator known from Z, which allows selective choice of the variables that are going to be provided in their primed form. The selective delta is mapped to the ordinary Z schema calculus as follows:

$$\Delta(list)S \mathrel{\widehat=} S \land (S \,\mathsf{n}\, (S \,\mathsf{n}\, (list))'$$

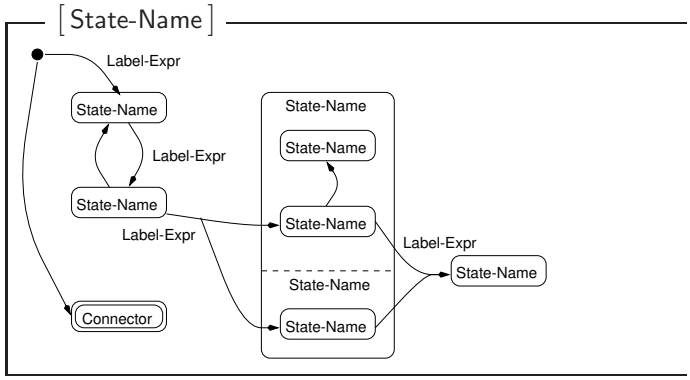
4 Specifying Operational Behavior

Operational-Paragraph	::=	Statechart
	|	Static-Reaction
	|	Transition
	|	Attribute

The operational behavior of a process class is defined by an Operational-Paragraph. $\mu\mathcal{SZ}$ supports Statemate-like statecharts and static reactions, abbreviations for transitions, and certain attributes for variables of the data space of a process class, which are used to control the combination of the axiomatic and operational semantics.

4.1 Statecharts

Statechart* ::=



State-Name	::=	Word
Connector	::=	Word
Label-Expr	::=	Trans-Name
	|	Predicate / Predicate

(*) The graphical statechart syntax is given as an example.

Statecharts describe the reactive behavior (control) of the process. They receive events and, depending on their current state, perform data operations, send events and change their state, i.e. perform state transitions.

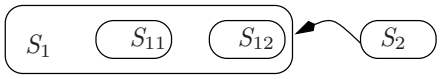
A statechart consists of states, transitions, labels and connectors. States are represented as boxes with rounded corners. A state is either a basic, an xor, or an and state. Boxes denoting basic states contain only a State-Name. And states are additionally partitioned with dashed lines. Boxes denoting an xor state contain a statechart. Boxes denoting an and-state contain a statechart in each partition. A statechart can contain default, history, deep history, or, fork and junction connectors. These connectors are described in [HP96, p. 4-13] in detail.

Transitions are labeled arrows that connect states and connectors. Default connectors may not be targets of a transition. Transitions may connect sub-states and connectors of different and or xor states (inter-level transition) but may not connect states or connectors that belong to different sub-states of the same and state. Transitions that enter (leave) and states can be forked (joint) such that they have one target (source) state for each sub-chart of the and state. Transitions are labeled by Label-Expr.

4.1.1 Referring to Statechart States from Z

For each process class an implicit Z type State is created from the statechart. State is defined as a free type of all state names. Sub-state names are preceded by their super-state's name followed by a colon; the colon in this form is part of the lexis of Word.

For a class with the following statechart, State would be defined as State ::= S_1 | S_2 | S_1:S_{11} | S_1:S_{12}.



In the axiomatic paragraphs of a process class, the instate_ relation can be used to test whether a process is in a given statechart state. Time-out conditions can be modelled by using the delay function, e.g. delay $S = 3$ means that the system entered state S three seconds ago.[3]

A data schema named CONTROL, declaring the variables instate, delay and now, is implicitly available in every class. The unary relation instate always contains the active states of the statechart and the function delay records for every state the duration since it was entered the last time. If a state S has not yet been entered, we have delay $S = 0$. The variable now gives the current system time.

DATA CONTROL

instate_ : ↑ State
delay_ : State ⤔ *TIME*
now : *TIME*

4.1.2 Controlling the Statechart History

In order to clear the history (history or deep-history symbols) of a statechart, there are two built-in event variables in every class: historyClear and deepClear. The action historyClear′ ↑ S clears the history of S, while deepClear′ ↑ S applies to S and all of its descendent states too. These events are collected in a data schema named HISTORY which is implicitly declared in each class:

DATA HISTORY

historyClear, deepClear : event State

4.1.3 Partitioning Statecharts

If several statecharts are given in a class, they are semantically united in a single chart. The gluing procedure works as follows. Top-level states with the same name are glued together. If one of these states is non-basic, the resulting state is also non-basic and has the same sub-chart. If both states are non-basic, the gluing procedure is continued recursively. In this case the states have to be compatible (e.g. both xor). Transitions are not glued together.

[3] In Statemate, one would use a time-out event for this purpose, e.g. $tm(en(S), 3)$

4.2 Label Expressions

A Label-Expr is either a transition name (see Section 4.5 on the next page) or an expression of the form *Guard* / *Action*, where both components are predicates. The context of the *Guard* predicate is the data space of the process class, the context of the *Action* is yielded by applying Z's Δ operator to the data space. This means that the dashed form of the variables is visible only in the *Action* part.

A transition is performed if its source states are active, the transition's guard is true and no transition with higher priority can be performed (see 4.7). As a proof obligation it remains to be shown that the precondition of a label's action always holds if the transition is taken. If invariants described by the axiomatic paragraphs of the process class are violated as the effect of executing a statechart's action, the specification is inconsistent.

4.3 Event Variables

$\mu\mathcal{SZ}$ supports event variables, which are related to the operational view. Event variables are used for realizing broadcast communication between processes. They hold an assigned value for one step only and are set to undefined after the step if they are not written again. Signals are event variables which do not carry values.

In the axiomatic view, there are three functions related to event variables: df tests whether an event variable has a defined value, vl returns the value, and with $\uparrow$ an event can be sent with a given value. Furthermore, there is an abbreviation for the type of signals as a special case of event variables.

$$
\begin{array}{|l}
\hline
[X] \\
\mathsf{df}\ _ : \mathbb{P}(\text{event}\ X) \\
\mathsf{vl} : \text{event}\ X \ \tilde{}\ X \\
_ \uparrow _ : \text{event}\ X \ \tilde{}\ X \\
\hline
(\mathsf{df}\ _) = \mathsf{dom}\,\mathsf{vl} \land \mathsf{vl} = (_ \uparrow _) \\
\hline
\end{array}
\qquad
\begin{array}{l}
[\text{event}\ X] \\
\mathsf{signal} == \text{event}\{\, 0 \,\}
\end{array}
$$

For example, a label expression which requires a signal a and generates a signal b in the next step is written $\mathsf{df}\ a\ /\ \mathsf{df}\ b'$. An event e : event $'$ with the value 4 is generated in the next step by writing $e' \uparrow 4$.

Event variables are subject to special treatment in the operational semantics: they are automatically set to be undefined in the initial state of the process and reset (set undefined) after each step, they are not written.

The event type constructor may only be used in type construction in the outermost position, e.g. event X # event Y is not permitted.

4.4 Static Reactions

Static-Reaction ::=

REACTION [State-Name]
Label-Expr
...
Label-Expr

| *REACTION* [State-Name] ≙ [Label-Expr; ...; Label-Expr]
| *REACTION* [State-Name] ≙ Transition-Name

Static reactions are bound to a state. They have the same syntax and the same context conditions as transitions, but they do not cause a state change. The static reactions of a state are evaluated and eventually executed when in the respective state. If no explicit State-Name is given, the static reaction is bound to the root state of the statechart. If no statechart is given for a class, only static reactions without an explicit state name are allowed. In this case, the static reactions of the class are (evaluated and eventually) executed in every step. If the guards of several Label-Exp are true simultaneously, their actions are executed in parallel, as if they belonged to parallel statecharts.

4.5 Transitions

Transition ::=

TRANS Trans-Name [Renaming-List]
Label-Expr

| *TRANS* Trans-Name [Renaming-List] ≙ [Label-Expr]

Trans-Name ::= Word

Transition declarations are used to abbreviate a label-expression for use in a statechart or a static reaction.

The label expression is treated similar as in statecharts: a guard predicate is checked in the context of the unprimed variables of the data space, whereas the action predicate may contain primed and unprimed variables.

4.6 Variable Attributes

Attribute ::= *INPUT* Decl-Name-List
| *INPUT* Schema-Name, ..., Schema-Name
| *DERIVED* Decl-Name-List

Attributes define properties of variables of the data space of a class which are relevant in context conditions of label expressions and their meaning in the operational step semantics. All the names listed in the Decl-Name-List must be part of the data space, or if Schema-Name is used, all variables declared in the references schemas must be part of the data space.

4.6.1 Input Variables

A variable attributed to be *INPUT* cannot be used in its primed form in the label expressions of a process class. This feature is used to mark variables of ports to be read-only, giving a strict syntax-based guarantee for analysis tools that the process class does not change the value of these variables.

4.6.2 Derived Variables

As an *INPUT* variable, a *DERIVED* variable cannot be used in its primed form in the label expressions of a process class. The values of derived variables are constrained by predicates and change automatically to fulfil these predicates. Derived variables do not contribute to the data space of a process class. Through the predicates, the values of derived variables depend on normal (primary) variables. The values do not have to be exactly determined by the predicates. They can be seen as given by a function, mapping the process class' data space to values. Thus it is ensured that even if the derived variable is not fixed in one run of a class' instantiation, the same assignment of data space variables leads to the same values of derived variables. Thus, derived variables can only change their value if normal variables they depend on, also change their values. Derived variables can be exported through ports as normal variables, but cannot be input variables.

Consider, for example, the data schema D. The variables y and z are supposed to change their values automatically, according to some given constraints. This is achieved by the declaration: *DERIVED* y, z.

DATA D

$x, y, z : \mathbb{E}$

$y = 2 * x$

$z > x$

The effect of this declarationsis that the variables y' and z' cannot be used in label expressions of a class. Whenever the value of x is changed, y and z are changed implicitly, according to the predicate.

4.7 An Outline of the Step Algorithm

We give a short explanation of the statecharts step algorithm. For a detailed description see [HN96]. The input for the step algorithm is the current process status (σ), e.g. the states the process currently resides in and a binding of the variables of the data space.

Using this information, the set of enabled transitions is computed. A transition is enabled if all of its source states are active and the guard of its label is true for the current variable bindings. From this set, a maximal non-conflicting[4] set is computed by removing those transitions that are in conflict with an enabled transition of higher priority[5], and non-deterministically removing a transition that has equal priority with a conflicting transition.

Each transition of the maximal set of non-conflicting transitions is executed. This is done by leaving all of the transition's source states and entering the target states. Since the transitions are non-conflicting, it is ensured that the new process status (σ') is a valid status.

Now the set of enabled static reactions can be computed. A static reaction of a state is enabled if the state is active at the beginning of the step, the state is not being exited by any of the

[4] Two transitions are said to be in conflict if the intersection of the set of states they are exiting is non-empty.

[5] A transition has a higher priority than another transition if its context is an ancestor of the other transition's context.

chosen transitions, and the guard of its label is evaluated to be true for the variable bindings at the beginning of the step.

In the next stage, all actions of the chosen transitions and static reactions are executed by updating the current variable bindings. It is important to note that for the execution of an action the variable bindings at the beginning of the step are used. Changes made in a step are visible only in the next step. If the value of an element is modified by more than one action in a step, a so-called racing condition occurs. Consider a variable binding $\langle x \Rrightarrow 1 \rangle$ at the beginning of a step and two actions (operations) to be executed in this step: $Op_1 \mathrel{\widehat{=}} [\Delta(x)\text{DATA} \mid x' = x + 1]$ and $Op_2 \mathrel{\widehat{=}} [\Delta(x)\text{DATA} \mid x' = x + 2]$. Then, as the result of executing the two operations in the same step, we get either $\langle x \Rrightarrow 2 \rangle$ or $\langle x \Rrightarrow 3 \rangle$, depending on which operation was executed first, but not $\langle x \Rrightarrow 4 \rangle$ because for both operations, the value of x at the beginning of the step is assumed.

The written variables are determined from the set of executed actions. A variable is written by an action if its primed version appears in the action's predicate or is declared in a referenced schema. Variables that are not written keep their value if they are normal (persistent) variables, whereas events are reset. Finally, the values of the derived variables are inferred from the new bindings of the primary variables.

Getting from one process state to the next one by either carrying out a set of consistently enabled transitions or by advancing the clock by a time unit is called a (state or time) step:

$$\sigma_0 \xrightarrow{T_0} \sigma_1 \ldots \sigma_i \xrightarrow{tick} \sigma_{i+1} \ldots \sigma_j \xrightarrow{T_k} \sigma_{j+1}$$

Here, the T_n denote maximal non-conflicting sets of enabled transitions. A time step is only executed if there are no more enabled transitions and static reactions in a statechart. A sequence of state steps between two time steps is called a super step.

5 Specifying Behavioral Constraints

Behavioral-Paragraph ::= [[Schema-Role] DYN Schema-Name
Decl-Part
Dyn-Axiom-Part]

Schema-Expr ::= ...
| Dyn-Schema-Text
Schema-Text ::= ...
| Dyn-Schema-Text
Dyn-Schema-Text ::= $[DYN$ Decl-Part [| Dyn-Axiom-Part]]
Dyn-AxiomPart ::= Dyn-Predicate; ...; Dyn-Predicate
Dyn-Predicate ::= Predicate
| Dyn-Predicate $\overset{\lceil \text{Dyn-Constr} \rceil}{\leadsto}$ Dyn-Predicate
| Dyn-Predicate ⦶ Dyn-Predicate
| Dyn-Predicate ⦸ Dyn-Predicate
| everywhere$_{\lceil \text{Dyn-Constr} \rceil}$ Dyn-Predicate
| somewhere$_{\lceil \text{Dyn-Constr} \rceil}$ Dyn-Predicate
| repeat$_{\lceil \text{Dyn-Constr} \rceil}$ Dyn-Predicate
| ⌈ Predicate ⌉
| Ψ Predicate Ω
| Dyn-Predicate$_{\text{Dyn-Constr}}$
| Dyn-Constr
Dyn-Constr ::= ℓ In-Rel Expr
| δ In-Rel Expr

The behavior of a process class can be restricted or even be completely specified using a version of discrete temporal interval logic which is embedded in Z. This extension of Z is called Dynamic Z and is described in [BG97]. The following explanations and definitions can be skipped by users of $\mu\mathcal{SZ}$ who do not intend to use Dynamic Z or temporal logic.

We give a short informal introduction to Dynamic Z. For the semantics of Dynamic Z, see [BG97]. The interplay between Dynamic Z and $\mu\mathcal{SZ}$ is discussed in the next section.

5.1 Dynamic Values and Dynamic Schemata

Dynamic Z adds to Z the concepts of dynamic values and dynamic schemata. A dynamic value models a trace of elements of some type and is represented in Z by the type

$$\mathrm{dyn}\, A == \{f : \mathbb{N}' \rightsquigarrow A \mid \forall i : \mathbb{N}' \bullet i + 1 \in \mathrm{dom}\, f \Rightarrow i \in \mathrm{dom}\, f\}$$

A dynamic schema bundles a collection of dynamic values. These values satisfy a property given in temporal interval logic (a dynamic predicate as defined in the next section). For example, the dynamic schema $S \mathrel{\widehat=} [DYN\, x, y : \mathrm{dyn}\, \mathbb{Z} \mid (\lceil x > 0 \rceil \not\emptyset \lceil x < 0 \rceil) \wedge \lceil y \geq x \rceil]$ describes the dynamic values x and y – traces of integers – such that x is greater than 0 up to some state and less 0 after this state, whereas y is greater than or equal to x in each state. For each binding $\sigma \in S$, $\sigma.x$ and $\sigma.y$ represent traces as described above. By definition, the traces of the components of a

binding of a dynamic schema have the same length.[6] Since dynamic schemata denote plain sets of bindings, they can be used wherever plain schemata can. For example, the schema expression $S \wedge [x, y : \mathrm{dyn}\,\mathbb{E} \mid x = y]$ conjuncts the dynamic schema S with a plain schema, which enforces the dynamic values x and y to be equal as a whole, meaning pointwise equality of their traces.

Dynamic Z allows "types" of dynamic values to be declared. For example, the set $X == \{DYN\ x : \mathrm{dyn}\,\mathbb{E} \mid \lceil x > 0 \rceil \emptyset \lceil x < 0 \rceil\}$ denotes the subset of $\mathrm{dyn}\,\mathbb{E}$ where traces behave as described for the example above. With this definition of X, an alternate description for the above-defined dynamic schema S would be $[DYN\ x : X;\ y : \mathrm{dyn}\,\mathbb{E} \mid \lceil y \geq x \rceil]$. This again is equivalent to $[DYN\ x, y : \mathrm{dyn}\,\mathbb{E} \mid x \in X \wedge \lceil y \geq x \rceil]$, as to be expected in a Z-based framework.

Any conventional Z schema (also in top-level Z paragraphs) or $\mu\mathcal{SZ}$ schema with role can be tagged to be dynamic, either by preceding the schema name with the keyword *DYN* in a schema box, or by preceding schema text with the keyword *DYN*.

5.2 Dynamic Predicates

Dynamic predicates are a superset of normal Z predicates (such as element test, conjunction, negation, universal quantification and its derivatives.)

State Observations The state predicate $\lceil P \rceil$ is satisfied for those traces where the normal Z predicate P is satisfied at each point. For example, $\lceil x > 0 \rceil$ holds for traces of x such that for every point i in the trace, $(x\,i) > 0$ holds (representing the trace of x as a function $\mathbb{N} \to \mathbb{E}$). In a state predicate a variable $x : \mathrm{dyn}\,A$ denotes an element of type A.

A second form of observing states is the transition predicate, which allows observations to be related at the beginning and the end of a trace: $\Psi x = x + 1\,\Omega$ describes those traces, where in the last state the value of x is by one greater than in the first.

Length Constraints A length constraint is denoted by the predicate $\ell\ \underline{R}\ E$, where the expression E must denote a natural number, and $\underline{R}$ is some binary relation on natural numbers. For example, $\ell \leq 2$ describes those traces whose length, in discrete steps, is less than or equal to 2. For convenience, we can also annotate length constraints at other dynamic predicates: $\lceil x \leq 0 \rceil_{\ell=1}$ abbreviates $\lceil x \leq 0 \rceil \wedge \ell = 1$.

Duration Constraints While length constraints address step numbers, duration constraints specify real-time. They are of the form $\delta\ \underline{R}\ E$, where E must be of type *TIME*. For example, $\delta \leq 4\mathsf{s}$ describes those traces whose duration, in real time, is less than or equal to 4 seconds. For convenience, we can also annotate duration constraints at other dynamic predicates: $\lceil x \leq 0 \rceil_{\delta \leq 4\mathsf{s}}$ abbreviates $\lceil x \leq 0 \rceil \wedge \delta \leq 4\mathsf{s}$.

Chopping The basic chopping operator of Dynamic Z is an overlapping chop, $D_1 \emptyset D_2$. It is satisfied for those traces which can be chopped at a state such that D_1 holds up to this state and D_2 holds from this state to the end. For example, $\lceil x > 0 \rceil \emptyset \Psi' x = x - 1\,\Omega_{\ell=2}$ describes those traces where x is constrained to be greater than zero in any state except the last, and in the last state its value is decremented by one compared to the previous state. The normal chop known from interval logics, $D_1 \emptyset D_2$, is a derived form, defined as $D_1 \emptyset true_{\ell=2} \emptyset D_2$.

Implication The temporal implication in interval logics, the leadsto predicate, has the form $D_1 \rightsquigarrow D_2$, and is an abbreviation for $D_1 \emptyset true \Rightarrow D_1 \emptyset D_2$. For example, $\lceil x > 0 \rceil_{\ell=1} \rightsquigarrow \Psi x =$

[6] Thus the bindings of a dynamic schemata are isomorphic to traces of bindings of the individual components (for the above example these would be the traces $\mathrm{dyn}([x, y : \mathbb{v}])$). Actually, the last representation seems to be more natural, but the chosen one lets Dynamic Z be pure syntactic sugar on top of Z, since it does not require a new semantic category for the meaning of dynamic schemata.

$x - 1 \,\Omega_{\ell=2}$ describes those traces where if in the first state x is greater than 0, then in the next step x is equal to the previous x minus 1 and the trace ends there. For convenience, it is possible to annotate the leadsto arrow with a length or a duration constraint: $D_1 \overset{\ell \underline{R} E}{\rightsquigarrow} D_2$ abbreviates $D_1 \rightsquigarrow \ell \; \underline{R} \; E \; \emptyset \; D_2$, and $D_1 \overset{\delta \underline{R} E}{\rightsquigarrow} D_2$ abbreviates $D_1 \rightsquigarrow \delta \; \underline{R} \; E \; \emptyset \; D_2$.

Quantification Dynamic Z entails the usual versions of existential and universal quantification of interval logics. For example, everywhere $(\ell > 1 \Rightarrow \Psi \, x < \lambda \Omega)$ describes those traces where the value of x increases monotonically. In general, everywhere D holds for traces such that for each sub-trace D holds, and somewhere D for traces such that there exists a sub-trace for which D holds. The repetition, repeat D, holds for traces such that $D \; \emptyset \; D \; \emptyset \; \ldots$ holds (one or more repetitions of D).

For convenience, it is possible to annotate the quantifiers with a length or a duration constraint: $\mathsf{everywhere}_{\ell \underline{\mathrm{R}} \mathrm{E}}$ D abbreviates everywhere $(\ell \; \underline{R} \; E \Rightarrow D)$, and $\mathsf{everywhere}_{\delta \underline{\mathrm{R}} \mathrm{E}}$ D abbreviates everywhere $(\delta \; \underline{R} \; E \Rightarrow D)$. Similar conventions apply to somewhere.

5.3 Conversions

Static schemata can be converted to dynamic schemata (trace widening) and vice versa (state narrowing). If a static schema is referenced in a dynamic context, all variables are lifted to dynamic variables and the static predicate is lifted to a state observation. Thus, the result of trace widening is the set of all traces of the declared variables, where the predicate holds for each point in each trace. Trace widening is, e.g., used if variables declared in a static data or port schema are constrained by a dynamic property schema.

State narrowing is the counterpart of trace widening. It changes all dynamic variables to static variables. The result of state narrowing is the set of all variable bindings that are reachable under the dynamic constraints. Note that widening and afterwards narrowing a static schema results in this schema, whereas if a dynamic schema is narrowed and then widened, all dynamic information is lost. More precisely, the following rules are applied for widening and narrowing:

- trace widening: every variable of type τ, such that there is no τ' with $\tau = \mathrm{dyn}\, \tau'$, is implicitly converted into type $\mathrm{dyn}\, \tau$. The property of an included static schema P is converted to the property $\lceil P \rceil$.
- state narrowing: every variable of type $\mathrm{dyn}\, \tau$ which is declared or included in a non-dynamic schema is implicitly converted into type τ. Roughly, the dynamic property is converted into a static property by changing "chop" to "or" and forgetting length constraints.

These rules imply that variables of the signature of a dynamic schema always have type $\mathrm{dyn}\, \tau$ and that variables of a conventional schema never have type $\mathrm{dyn}\, \tau$. Since the schema text appearing in set comprehensions and elsewhere is by default conventional, the extended syntax for schema text must be used in such a context to specify dynamic values (as for example in $\mathrm{const}\, A == \{DYN \; x : A \mid \exists c : A \bullet \lceil x = c \rceil\}$, which denotes the set of dynamic values over A which are constant in each trace.)

5.4 Dynamic Data Space of a Process Class

The data space of a process class is interpreted as a dynamic schema as follows. Let D_s, P_s, I_s and A_s denote the non-dynamic *DATA*, *PORT*, *INIT* and *PROPERTY* schemas of a class, and D_d, P_d, I_d and A_d the dynamic ones:

DYN DATA

D_s; D_d; P_s; P_d

$\lceil I_s \rceil_{\ell=1}$ ∅ *true*; I_d ∅ *true*; $\lceil A_s \rceil$; A_d

The conversions between dynamic variables allow selective use of dynamic schemas in a process class, depending on the methodology. For example, one can specify

DATA D

x, y : Æ

PROPERTY DYN D_behavior

D

$(\lceil x > 0 \rceil$ ∅ $\lceil x < 0 \rceil) \wedge \lceil y \geq x \rceil$

which is equivalent to

DATA DYN D

x, y : Æ

$(\lceil x > 0 \rceil$ ∅ $\lceil x < 0 \rceil) \wedge \lceil y \geq x \rceil$

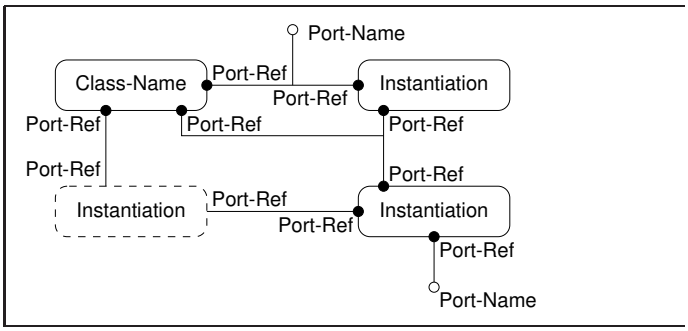
6 Specifying Structural Embedding

Structural-Paragraph	::=	Configuration
	|	Enrichment

The relation of a process with its environment is described by a Structural-Paragraph.

6.1 Configurations

Configuration* ::=



Instantiation	::=	Inst-Name : Class-Name [Gen-Actuals]
Port-Ref	::=	Schema-Ref
Port-Name	::=	Schema-Name

(*) The graphical configuration syntax is given as an example.

The instances of a process class can aggregate instances of other classes, or require that associated instances do exist in the environment of every instance of the process class. This, together with the communication topology of aggregated and associated processes, is described by a Configuration.

A Configuration consists of a set of instantiations of process classes and communication links between them. Aggregated instances are denoted by a solid box, associated by dashed boxes. The process which is an instance of the currently specified class is denoted by a box labelled simply with the class-name. Within a configuration, all instantiated processes have unique names.

A port of a process is denoted by a dot at the process' box, labelled with the port name. Communication links are established by connecting ports by drawing a line between two or more ports. Internal ports are denoted by black circles, external ports by white circles. If a process class has a configuration, the (external) visibility of ports is given by all external ports declared in this configuration. Otherwise, if no configuration is given, all ports are implicitly external. External ports might also be used to establish internal communication associations.

Communication between processes is established by plugging ports of two or more processes together. Variables with the same name are identified (i.e. always have the same value). However, renaming port variables in the configuration of a process class does not cause a variable renaming in the whole process class. The renaming in the configuration simpy establishes the above-mentioned equality of values.

Ports may be joined in a configuration and a new name might be given, i.e. two ports of two aggregates of a process are joined and declared as port of the aggregating class.

A process can also define associations to other processes. An association is a requirement on the context of each instance of the class, i.e. if a class *C* links its port *CPort* to the *DPort* of an

associated class D, then wherever C is instantiated a corresponding association has to be present.

6.2 Enrichment

Enrichment ::= *ENRICH* Class-Name [Gen-Actuals]

A class can enrich other classes. The class then inherits all components of the enriched class and may extend them. Enrichment is a syntactical operation and components with same name are identified.

Schemas with the same name are conjoined, i.e. the signatures are joined to form the signature of the resulting schema and the predicates are conjoined. The signature of the schemas must be compatible. If the classes have statecharts, the resulting statechart is formed according to the rules of statechart partitioning (see Section 4.1 on page 8). Additionally, the configurations of the enriched classes are merged with the configuration of the process class.

A LaTeX Syntax for $\mathcal{SZ}$

A.1 Specification

Specification	::=	(Process-Class | Paragraph)$*$

A.2 Classes

Process-Class	::=	`\begin{class}{` Class-Name [`[` Gen-Formals `]`] `}`
		(Class-Paragraph [`\\`])$*$
		`\end{class}`

Class-Paragraph	::=	`\begin{zed}` Class-Item$*$ `\end{zed}`
	|	Definition

	|	`\begin{schema}{` [Schema-Role] [`\Dyn`] Schema-Name `}`
		Decl-Part [`\where` Axiom-Part]
		`\end{schema}`

	|	Statechart-Box
	|	Static-Reaction
	|	Transition-Box
	|	Configuration-Box
Class-Item	::=	[Schema-Role] Schema-Name `\defs` Schema-Expr
	|	`\Enrich` Class-Name [Gen-Actuals]
	|	`\Input` Name-Decl-List
	|	`\Input` Schema-Name `,` ... `,` Schema-Name
	|	`\Derived` Name-Decl-List
	|	Abbreviation
Schema-Role	::=	`\Data` | `\Port` | `\Property` | `\Init` | `\Schema` | Role-Alias

A.3 Z Paragraphs

```
Paragraph      ::= Definition
                 | Abbreviation
                 | Schema-Box
Schema-Box     ::= \begin{schema}{ [ \Dyn ] Schema-Name [ [Gen-Formals] ] }
                       Decl-Part [ \where Axiom-Part ]
                   \end{schema}
Definition     ::= \begin{axdef} [ [Gen-Formals] ]
                       Declaration [ \where Axiom-Part ]
                   \end{axdef}
                 | \begin{gendef} [Gen-Formals]
                       Declaration [ \where Axiom-Part ]
                   \end{gendef}
Abbreviation   ::= Def-Lhs == Expression
                 | Ident ::= Branch | ... | Branch
                 | [ Ident , ... , Idents ]
                 | Schema-Name [ Gen-Formals ] \defs Schema-Expr
Decl-Part      ::= Basic-Decl Sep ... Sep Basic-Decl
Axiom-Part     ::= Predicate Sep ... Sep Predicate
Sep            ::= ; | \\
Branch         ::= Ident
                 | Var-Name \ldata Expression \rdata
Def-Lhs        ::= Var-Name [ Gen-Formals ]
                 | Pre-Gen Decoration Ident
                 | Ident In-Gen Decoration Ident
```

A.4 Schemata

```
Schema-Exp     ::= \forall Schema-Text @ Schema-Exp
                 | \exists Schema-Text @ Schema-Exp
                 | \exists_1 Schema-Text @ Schema-Exp
                 | [ Schema-Text ]
                 | Schema-Ref
                 | \lnot Schema-Exp
                 | Schema-Exp Junctor Schema-Exp
                 | Schema-Exp \project Schema-Exp
                 | Schema-Exp \hide (Decl-Name , ... , Decl-Name)
                 | Schema-Exp \semi Schema-Exp
                 | Schema-Exp \pipe Schema-Exp
                 | (Schema-Exp)
Schema-Text    ::= [ \Dyn ] Declaration [ | Predicate ]
Schema-Ref     ::= [ Delta-Xi ] Schema-Name Decoration [ Gen-Actuals ] [ Renaming ]
Delta-Xi       ::= \Delta [ (Decl-Name , ... , Decl-Name) ]
                 | \Xi
Renaming       ::= [ Decl-Name/DeclName , ... , Decl-Name/Decl-Name ]
Declaration    ::= Basic-Decl ; ... ; Basic-Decl
Basic-Decl     ::= Decl-Name , ... , Decl-Name : Expression
                 | Schema-Ref
                 | [ Schema-Text ]
```

A.5 Predicates

Dynamic and ordinary predicates are not differentiated here so that the TEX-syntax remains small.

Predicate	::=	\forall Schema-Text @ Predicate
	|	\exists Schema-Text @ Predicate
	|	\exists_1 Schema-Text @ Predicate
	|	\LET Let-Def ; . . . ; LetDef @ Predicate
	|	Expression Rel . . . Rel Expression
	|	Pre-Rel Decoration Expression
	|	Schema-Ref
	|	\pre Schema-Ref
	|	true | false
	|	\lnot Predicate
	|	Predicate Junctor Predicate
	|	Predicate \dleadsto Predicate
	|	Predicate \dleadstol{In-Rel}{Expression} Predicate
	|	Predicate \dleadstod{In-Rel}{Expression} Predicate
	|	Predicate \dseq Predicate
	|	Predicate \dchop Predicate
	|	\deverywhere Predicate
	|	\deverywherel{In-Rel}{Expression} Predicate
	|	\deverywhered{In-Rel}{Expression} Predicate
	|	\dsomewhere Predicate
	|	\dsomewherel{In-Rel}{Expression} Predicate
	|	\dsomewhered{In-Rel}{Expression} Predicate
	|	\drepeat Predicate
	|	Predicate \dLength{ In-Rel }{Expression}
	|	Predicate \dDuration{ In-Rel }{Expression}
	|	\dlength In-Rel Expression
	|	\ddurationIn-Rel Expression
	|	\dlstate Predicate \drstate
	|	\dltrans Predicate \drtrans
	|	(Predicate)
Junctor	::=	\land | \lor | \implies | \iff
Rel	::=	= | \in | In-Rel Decoration
Let-Def	::=	Var-Name == Expression

A.6 Expression

Expression ::=
- \lambda Schema-Text @ Expression
- \mu Schema-Text [@ Expression]
- \LET Let-Def ; ... ; Let-Def @ Expression
- \IF Predicate \THEN Expression \ELSE Expression
- Expression In-Gen DecorationExpression
- Expression \cross ... \cross Expression
- Expression In-Op DecorationExpression
- \power Expression
- Pre-GenDecoration Expression
- - Decoration Expression
- Expression \limg Expression \rimg Decoration
- Expression Expression
- Var-Name [Gen-Actuals]
- Number
- Schema-Ref
- \{ [Expression , ... , Expression] \}
- \{ Schema-Text [@ Expression] \}
- \langle [Expression , ... , Expression] \rangle
- \lbag [Expression , ... , Expression] \rbag
- (Expression , ... , Expression)
- \theta Schema-NameDecoration [Renaming]
- Expression . Expression
- Expression Post-Op Decoration
- Expression ^ { Expression }
- (Expression)
- \dlength
- \dduration

A.7 Identifier

```
Word            ::= \ Letter Letter*
                  | Letter (Letter | Digit | \: | \_)*
Letter          ::= A | ... | Z | a | ... | z
Digit           ::= 0 | ... 9
Stroke          ::= ' | _Digit | ? | !
Ident           ::= Word Decoration
Decl-Name       ::= Word | Op-Name
Var-Name        ::= Word | (Op-Name)
Op-Name         ::= \_ In-Sym Decoration \_
                  | Pre-Sym Decoration \_
                  | \_ Post-Sym Decoration
                  | \_ \limg \_ \rimgDeocration
                  | - Decoration
Pre-Sym         ::= Pre-Gen | Pre-Rel
Post-Sym        ::= Post-Op
Decoration      ::= [ Stroke ... Stroke ]
Gen-Formals     ::= [Ident , ... , Ident]
Gen-Actuals     ::= [Expression , ... , Expression]
Number          ::= Digit(Digit)*
Class-Name      ::= Word
Schema-Name     ::= Word
Transition-Name ::= Word
State-Name      ::= Word
Pre-Rel         ::= Word
In-Rel          ::= Word
Post-Rel        ::= Word
Pre-Op          ::= Word
In-Op           ::= Word
Post-Op         ::= Word
Role-Alias      ::= Word
```

A.8 Statecharts

```
Label-Expr      ::= Transition-Name
                  | [ Predicate ] [ \tr Predicate ]
Statechart-Box  ::= \begin{State-Type} [ { [ State-Name ] } ]
                      State-Decls\where Transition ... Transition
                    \end{State-Type}
State-Type      ::= xorstate | andstate
State-Decls     ::= Word , ... , Word : (\State | \Connector)
Transition      ::= Word \When Label-Expr \Goto Word
Static-Reaction ::= \begin{reaction}{ [ State-Name ] }
                      Label-Expr Sep ... Sep Label-Expr
                    \end{reaction}
                  | \Reaction State-Name \defs
                    [ Label-Expr Sep ... Sep Label-Expr]
Transiton-Box   ::= \begin{trans}{Transition-Name}
                      Label-Expr
                    \end{trans}
                  | \Trans Transition-Name \defs
                    [ Label-Expr]
```

A.9 Configurations

Configuration-Box ::= `\begin{config}`
Config-Decls `\where` Data-Flows
`\end{config}`

B LaTeX Symbols

Logic					
¬	\lnot	unary	pre	\pre	unary
∧	\land	left	∨	\lor	left
⇒	\implies	right	⇔	\iff	left
∀	\forall		∃	\exists	
⊖	\project	left	⊓	\hide	left
⨾	\semi	left	≫	\pipe	left
if	\IF		**then**	\THEN	
else	\ELSE		**let**	\LET	

Sets					
↑	\power	pregen	↓	\finset	pregen
∅	\emptyset	word	#	\#	word
∩	\cap	inop 4	∪	\cup	inop 4
\	\setminus	inop 3			
⊆	\subseteq	inrel	⊂	\subset	inrel
=	=	inrel	≠	\neq	inrel
∈	\in	inrel	∉	\notin	inrel
⋃	\bigcup	word	⋂	\bigcap	word
disjoint	\disjoint	prerel	partition	\partition	inrel

Relations and Functions					
↦	\mapsto	inop 1	×	\cross	inop 1
dom	\dom	word	ran	\ran	word
∘	\circ	inop 4	⨾	\comp	inop 5
⊕	\oplus	inop 5	~	\inv	postop
⩤	\dres	inop 6	⩥	\rres	inop 6
⨅	\ndres	inop 6	⨆	\nrres	inop 6
+	\plus	postop 6	*	\star	postop
⇸	\rel	ingen	⤔	\pfun	ingen
→	\fun	ingen	⤖	\pinj	ingen
⤖	\inj	ingen	⤀	\psurj	ingen
⤀	\surj	ingen	⤖	\bij	ingen
⇸	\ffun	ingen	⇸	\finj	ingen
id	\id	word			
⦇	\limg		⦈	\rimg	

Integers					
ℙ	\num	word	ℕ	\nat	word
..	\upto	inop 2			

$+$	`+`	inop 3	$-$	`-`	inop 3
$*$	`*`	inop 4	div	`\div`	inop 4
mod	`\mod`	inop 4			
$<$	`<`	inrel	$\leq$	`\leq`	inrel
$>$	`>`	inrel	$\geq$	`\geq`	inrel

Sequences

$\langle$	`\langle`		$\rangle$	`\rangle`	
seq	`\seq`	pregen	iseq	`\iseq`	pregen
in	`\inseq`	inrel	$\emptyset$	`\cat`	inop 3
prefix	`\prefix`	inrel	suffix	`\suffix`	inrel
$\ominus$	`\filter`	word	$\sqcup$	`\extract`	word

Bags

¡	`\lbag`		¿	`\rbag`	
bag	`\bag`	pregen	Φ	`\inbag`	inrel
$\sqsubseteq$	`\subbageq`	inrel	$\sharp$	`\bcount`	inop 5
$\uplus$	`\uplus`	inop 3	!	`\uminus`	inop 3
$\otimes$	`\otimes`	inop 4			

Control

instate	`\instate`	prerel	delay	`\delay`	word
historyClear	`\historyClear`	word	deepClear	`\deepClear`	word

Events

event	`\event`	pregen	signal	`\signal`	word
df	`\df`	prerel	vl	`\vl`	word
$\uparrow$	`\emit`	inrel			

Schema Roles

Data	`\Data`	role	*Port*	`\Port`	role
Init	`\Init`	role	*Op*	`\Op`	role
Trans	`\Trans`	role	*Guard*	`\Guard`	role
Property	`\Property`	role	*Assertion*	`\Assertion`	role
DATA	`\DATA`	word	PORT	`\PORT`	word
INIT	`\INIT`	word	DATASPACE	`\DATASPACE`	word
PROPERTY	`\PROPERTY`	word	ASSERTION	`\ASSERTION`	word

C Library

The library can be considered to be an extension of Z's matematical tool kit [Spi92].

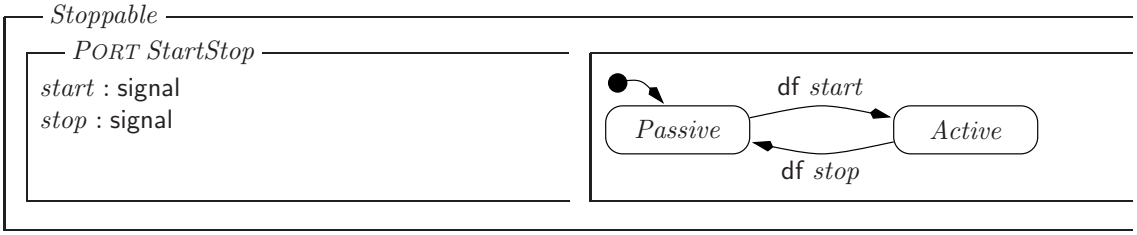
C.1 Booleans

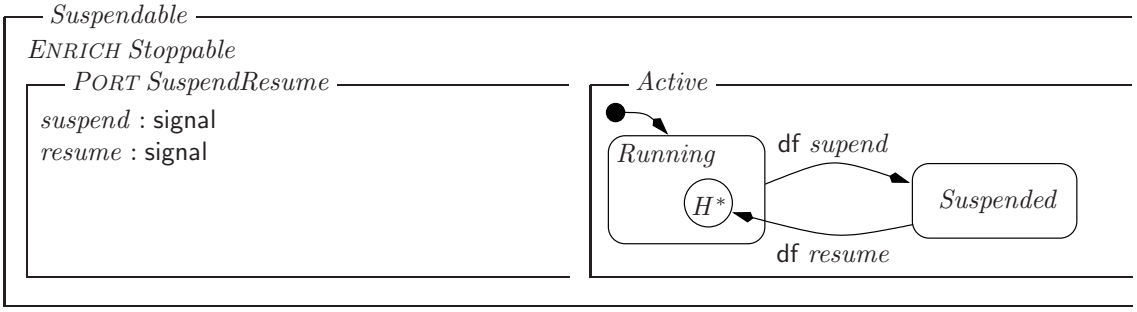
$\mathbb{B} ::= False \mid True$

C.2 Starting and Stopping Processes

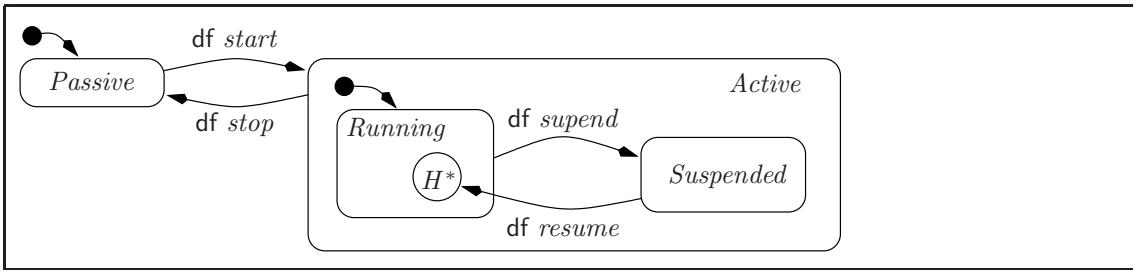
It is often desirable to start and stop a process or to suspend it. This behavior is implemented by the abstract process classes *Stoppable* and *Suspendable*. A suspendable process is also stoppable. A stoppable process is stopped by sending the event *stop* and started by sending *start*. These processes are used by enriching them and refining the state *Active* or *Running*.

Stoppable is a process with two elementary states representing the states of an abstract stoppable process, i.e. *Active* and *Passive*. By using events declared in the port *StartStop*, state changes can be performed.





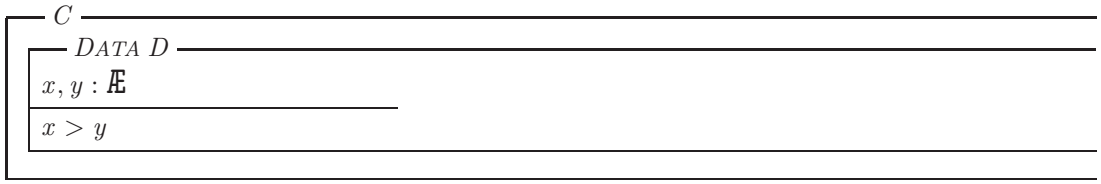
Suspendable defines an abstract suspendable process. By using its events declared in the port *SuspendResume*, the process can be suspended or resumed. *Suspendable* enriches *Stoppable*, refining the *Active* state, and therefore a suspendable process can also be started and stopped. The resulting statechart of *Suspendable* would be:



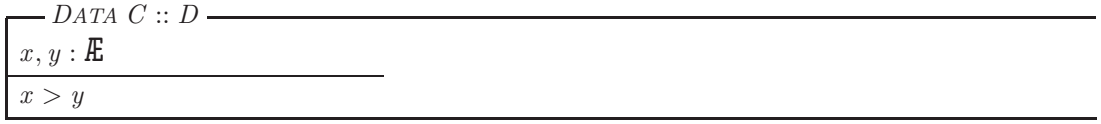
Suspendable is also used via enrichment and state refining, and for most cases the state *Running* will be specialized defining the behavior of the concrete process.

D Alternative External Representations

$\mu\mathcal{SZ}$ supports another representation of schemas avoiding "double boxes", which might be more appropriate. Consider the following example:



$\mu\mathcal{SZ}$ allows the class name to be placed in front of a schema, i.e. the above example could be written as:



E Using Statemate

The operational behavior as well as the configuration of a process class can be specified with the Statemate tool [HLN+90]. The ESPRESS tool environment provides support to translate Statemate models into $\mu\mathcal{SZ}$ and thus supplement a $\mu\mathcal{SZ}$ specification with Statemate specifications.

E.1 Configurations

Statemate activity charts can be used to describe $\mu\mathcal{SZ}$ configurations. In LaTeX syntax, for example, an activity chart can be referenced with the special Class-Paragraph `\config{`Chart-Name`}`. In the activity chart, off-page subactivities are interpreted as aggregated processes and external activities as associated processes. A control activity describes the class' operational behavior. Communication links (port binding) is denoted by information flows. The name of the information flow denotes the port name to be bound, i.e. if two processes are connected by an information flow P, their ports P are bound. This implies that both processes have a port P. External ports are denoted by information flows to external activities. Port bindings of the class itself are denoted by flows to or from its control activity, i.e. its statechart.

E.2 Operational Behavior

Operational behavior (statecharts and static reactions) can be described using Statemate. The full language is supported. Note, however, that particular tools might restrict the language (e.g. disallow history connectors). In order to use starting, stopping, suspending and resuming of a process, the process has to enrich *Stoppable* or *Suspendable*, as presented in C.2. In LaTeX syntax, Statemate statecharts can be referenced by the special Class-Paragraph `\chart{`Chart-Name`}`. Note that an activity chart of the class might have a control activity. This control activity always denotes the root statechart.

E.3 Statemate Label Expressions

A subset of the Statemate textual expression language is supported [HP96]. Generally, it is intended that guards and actions are specified using pure $\mu\mathcal{SZ}$ and the Statemate textual language is only used for simple logical operators such as and, or, or not. Guards and actions, specified in $\mu\mathcal{SZ}$, guard and op schemata are referenced as subroutines in Statemate[7]. Guards should be declared as functions with condition return type, and actions should be procedures. Both have no parameters, since guards and actions use the class' data space rather then explicit parameters.

Only a restricted part of the Statemate textual expression language is supported. In particular, loops and context variables (variables prefixed by `$`), bit operations, arrays, records, subroutines (specified in Statemate) and queues are not supported.

[7] Subroutines are introduced in Statemate Magnum 1.2 and documented in the updated version of [HP96] shipped with the release.

```
Stm-Label          ::=  [ Stm-Event-Expr ] [ [Stm-Cond] ] [ /Stm-Action ]
Stm-Event-Expr     ::=  Stm-Event-Name
                     |  tm(en(Stm-State-Name),Stm-Expr)
Stm-Cond           ::=  Stm-Cond and Stm-Cond
                     |  Stm-Cond or Stm-Cond
                     |  not Stm-Cond
                     |  in(Stm-State-Name)
                     |  true | false
                     |  Stm-Var-Name
                     |  Stm-Guard-Name()
                     |  Stm-Expr Stm-Inrel Stm-Expr
                     |  (Stm-Cond)
Stm-Inrel          ::=  = | /= | < | > | <= | >=
Stm-Expr           ::=  Stm-Expr Stm-Inop Stm-Expr
                     |  Stm-Var-Name
                     |  Integer
                     |  - Stm-Expr
                     |  (Stm-Expr)
Stm-Inop           ::=  + | - | * | /
Stm-Action         ::=  Stm-Event-Name
                     |  Stm-Op-Name ()
                     |  Stm-Action ; Stm-Action
                     |  Stm-Var-Name:=(Stm-Expr | Stm-Cond)
                     |  (tr! | fs!)(Stm-Var-Name)
                     |  (st! | sp! | sd! | rs!)(Stm-Instance-Name)
                     |  if Stm-Cond then Stm-Action [ else Stm-Action ] end if
                     |  when Stm-Event-Expr...
Stm-State-Name     |  Stm-Name.....Stm-Name
Stm-Var-Name       |  Var-Name
Stm-Guard-Name     |  Var-Name
Stm-Op-Name        |  Var-Name
Stm-Event-Name     |  Var-Name
Stm-Var-Name       |  Stm-Letter(Stm-Letter | Stm-Number | _)*
Stm-Letter         |  A...Z
Stm-Number         |  0...9
```

References

[BDG+96] Robert Büssow, Heiko Dörr, Robert Geisler, Wolfgang Grieskamp, and Marcus Klar. $\mu\mathcal{SZ}$ – Ein Ansatz zur systematischen Verbindung von Z und Statecharts. Technical Report 96-32, Technische Universität Berlin, Fachbereich Informatik, February 1996.

[BG97] Robert Büssow and Wolfgang Grieskamp. Combining Z and temporal interval logics for the formalization of properties and behaviors of embedded systems. In R. K. Shyamasundar and K. Ueda, editors, *Advances in Computing Science – Asian '97*, volume 1345 of *LNCS*, pages 46–56. Springer-Verlag, 1997.

[BGK97] R. Büssow, R. Geisler, and M. Klar. Spezifikation eingebetteter Steuerungssysteme mit Z und Statecharts. In *Tagungsband zur 5. Fachtagung Entwurf komplexer Automatisierungssysteme*. TU Braunschweig, 1997.

[BGK98] R. Büssow, R. Geisler, and M. Klar. Specifying safety-critical embedded systems with statecharts and Z: A case study. Accepted for publication at ETAPS'98, 1998.

[BGKM97] R. Büssow, R. Geisler, M. Klar, and S. Mann. Spezifikation einer Lichtsignalanlagen-Steuerung mit $\mu\mathcal{SZ}$. Technical Report 97-13, Technische Universität Berlin, Fachbereich Informatik, 1997.

[CW97] M. Conrad and D. Wetter. An adaptive cruise control system – a case study for the industrial use of formal methods. In *Proceedings of the First Workshop on Formal Design of Safety Critical Embedded Systems*, 1997.

[GHD98] Wolfgang Grieskamp, Maritta Heisel, and Heiko Dörr. Specifying safety-critical embedded systems with statecharts and Z: An agenda for cyclic software components. Accepted for publication at ETAPS'98, 1998.

[Guh97] Markus Guhe. Formale Spezifikation und systematische Implementierung der Steuerung einer Lichtsignalanlage in $\mu\mathcal{SZ}$. Diplomarbeit, Technische Universität Berlin, 1997.

[Har87] David Harel. Statecharts: A visual formalism for complex systems. *Science of Computer Programming*, 8(3):231–274, June 1987.

[HLN+90] David Harel, Hagi Lachover, Amnon Naamad, Amir Pnueli, Michal Politi, Rivi Sherman, Aharon Shtull-Trauring, and Mark Trakhtenbrot. Statemate: A working environment for the development of complex reactive systems. *IEEE Transactions on Software Engineering*, 16 No. 4, April 1990.

[HN96] David Harel and Amnon Naamad. The STATEMATE semantics of statecharts. *ACM Trans. Soft. Eng. Method.*, October 1996.

[HP96] David Harel and Michael Politi. Modeling reactive systems with statecharts: The statemate approach. i-Logix Inc, Three Riverside Drive, Andover, MA 01810, USA, June 1996. Part No. D-1100-43, 6/96.

[Spi92] J. M. Spivey. *The Z Notation: A Reference Manual*. Prentice Hall International Series in Computer Science, 2nd edition, 1992.

Index

Attribute, 11

Behavioral-Paragraph, 14

Class-Name, 3
Class-Paragraph, 3
Configuration, 18
Connector, 8

Data-Paragraph, 5
Dyn-AxiomPart, 14
Dyn-Constr, 14
Dyn-Predicate, 14
Dyn-Schema-Text, 14

Enrichment, 19

Instantiation, 18

Label-Expr, 8

Operational-Paragraph, 8

Port-Ref, 18
Process-Class, 3

Role-Name, 5

Schema-Expr, 14
Schema-Role, 5
Schema-Text, 14
Specification, 3
State-Name, 8
Static-Reaction, 11
Stm-Action, 31
Stm-Cond, 31
Stm-Event-Expr, 31
Stm-Event-Name, 31
Stm-Expr, 31
Stm-Guard-Name, 31
Stm-Inop, 31
Stm-Inrel, 31
Stm-Label, 31
Stm-Letter, 31
Stm-Number, 31
Stm-Op-Name, 31
Stm-State-Name, 31
Stm-Var-Name, 31
Structural-Paragraph, 18

Trans-Name, 11
Transition, 11